NEXT ENGINEER · _SHEERAT_

Software Engineering

BCS601

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, CS, IT | **Semester:** 6 | **Credits:** 4 | **L-T-P:** 3-1-0

Scheme: AKTU B.Tech 3rd year, Core, 2024-25 onwards

UNIT 1 Introduction and Software Development Life Cycle Models

UNIT 2 Software Requirement Specifications and Quality Assurance

UNIT 3 Software Design and Metrics

UNIT 4 Software Testing

UNIT 5 Software Maintenance and Project Management

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: models, processes and comparison tables.
3. **Solved examples**: the numericals that appear (function points, Halstead metrics, cyclomatic complexity, COCOMO, test case design).
4. **Formulas / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (software characteristics, crisis, SDLC models with diagrams) and Unit 2 (requirement engineering, DFD, ER diagram, decision table, SRS, SQA, ISO 9000, SEI-CMM).
- **Afternoon (3 hours):** Unit 3 (design concepts, coupling and cohesion, structure charts, metrics: LOC, Halstead, function points, cyclomatic complexity) and Unit 4 (testing levels, white-box and black-box, stubs and drivers, alpha and beta, walkthrough and inspection).
- **Evening (2 hours):** Unit 5 (maintenance types, re-engineering, configuration management, COCOMO, risk management). Then the "Last-hour checklist".
- **In the exam:** draw the model diagram first (waterfall, spiral, V-model, CFG), then explain phases, advantages and disadvantages. For numericals, write the formula, the table of values and the final answer with units.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction and Software Development Life Cycle Models

✍️ 2-mark definitions

- **Software:** computer programs together with the documentation and data needed to install, use and maintain them.
- **Software engineering (IEEE):** the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.
- **Software components:** programs (source and object code), documentation (requirements, design, test, user manuals) and operating procedures (installation, setup, operation).
- **Software crisis:** the problems seen from the 1960s onwards: projects late, over budget, unreliable, hard to maintain, and not meeting user needs, caused by growing size and complexity without disciplined methods.
- **Software process:** a set of activities (specification, design and implementation, validation, evolution) that produce software.
- **Software process model (SDLC model):** a simplified representation of a software process showing the order of phases and how they connect.
- **Legacy software:** old software that is still in use and critical to the business, but hard to change.
- **Software quality:** conformance to stated requirements and to implicit expectations such as reliability and usability.
- **Prototype:** an early, partial working version of the system built to clarify requirements.
- **Risk:** an uncertain event that, if it happens, has a negative effect on the project.

💡 Core concepts

Characteristics of software (how it differs from hardware)

1. Software is developed (engineered), not manufactured; the cost is in design, not production.
2. Software does not wear out, but it deteriorates because of changes (the "idealized" failure curve is flat; the "actual" curve has spikes after every change).
3. Most software is custom-built, though reuse of components is growing.
4. Software is complex, invisible (hard to visualize) and easy to change, which makes changes risky.

Software engineering vs conventional engineering

Point	Conventional engineering	Software engineering
Product	Physical, visible	Logical, invisible
Cost	Mostly in manufacturing and materials	Mostly in design and development (people)
Wear	Wears out	Does not wear out; deteriorates with changes
Laws	Based on laws of physics	No physical laws; based on logic and practice
Changes	Hard and costly after manufacture	Seem easy, which leads to frequent changes

Point	Conventional engineering	Software engineering
Maturity	Centuries old, well-established standards	Young discipline, standards still evolving

Software quality attributes (McCall's factors)

- Product operation: correctness, reliability, efficiency, integrity (security), usability.
- Product revision: maintainability, flexibility, testability.
- Product transition: portability, reusability, interoperability.
- ISO 9126 / ISO 25010 groups quality as functionality, reliability, usability, efficiency, maintainability and portability.

SDLC phases: feasibility study → requirement analysis and specification → design → coding and unit testing → integration and system testing → deployment → maintenance.

Waterfall model (classical / linear sequential)

- Phases follow one after another; each phase ends with a document that is reviewed and frozen before the next starts. The iterative waterfall adds feedback paths to earlier phases to correct errors.
- **Advantages:** simple, easy to manage, clear milestones and documentation; suits small projects with stable, well-understood requirements.
- **Disadvantages:** requirements must be known at the start; a working product appears only at the end; errors found late are costly; no room for customer feedback.

Prototyping model

- Gather initial requirements → quick design → build prototype → customer evaluates → refine requirements → repeat until the customer is satisfied → throw away the prototype and develop the real system (often with waterfall).
- **Use when:** requirements are unclear, or the user interface or a technical approach must be tested.
- **Advantages:** better understanding of requirements, early user involvement, fewer missing features. **Disadvantages:** extra cost of the prototype, customer may want the prototype itself delivered, poor design choices made for speed may remain.

Spiral model (Barry Boehm, 1986)

- Each loop of the spiral is a phase with four quadrants: (1) determine objectives, alternatives and constraints; (2) evaluate alternatives, identify and resolve risks (prototypes); (3) develop and verify the next-level product; (4) plan the next phase (customer review).
- The radius shows cumulative cost; the angle shows progress.
- **Called a meta-model** because it can include waterfall, prototyping and incremental models.
- **Advantages:** explicit risk analysis, good for large, high-risk projects, early prototypes. **Disadvantages:** needs risk-analysis expertise, costly, not suitable for small projects, hard to define end of the spiral.

Evolutionary model

- The system is built in successive versions; each version adds functionality and is delivered to users, whose feedback shapes the next version. Requirements are refined as the product evolves.
- **Advantages:** users get working software early; requirements can change. **Disadvantages:** hard to divide the system into increments; structure can degrade without good design.

Iterative enhancement (incremental) model

NEXT ENGINEER _sheerat_

- Requirements are divided into increments by priority. Each increment goes through a mini-waterfall (analysis, design, code, test) and is delivered. The first increment is the core product.
- **Advantages:** early delivery of core features, easier testing of small parts, lower initial cost, risk spread out. **Disadvantages:** needs good planning and a complete definition of the whole system to divide it; total cost may be higher.

Comparison of SDLC models

Model	Requirements	Risk handling	Customer involvement	Best for
Waterfall	Fixed and clear at start	Poor	Low (start and end)	Small, well-understood projects
Prototype	Unclear	Medium	High	Projects with unclear requirements or new UI
Spiral	Can change	Very good (explicit)	High at each loop	Large, high-risk projects
Evolutionary	Evolve over time	Medium	High	Projects where users learn by using
Incremental	Mostly known, delivered in parts	Medium	Medium to high	Products needing early core delivery

🧮 Solved examples

Example 1 (choosing a model).

- A bank wants a payroll system with requirements fixed by law and a known team: **Waterfall**.
- A startup is unsure what its mobile app screens should look like: **Prototype**.
- A defence missile-control system with new technology and high risk: **Spiral**.
- An online store that must launch with basic shopping in three months and add reviews and recommendations later: **Incremental**.

Example 2 (cost of fixing errors late). If an error costs ₹1,000 to fix in the requirements phase, a common rule of thumb says it costs roughly 5 to 10 times more in design and coding and up to 100 times more after delivery. This is why reviews of the SRS and design are cheaper than finding the bug in production.

Key points to remember

- Software is engineered, not manufactured; it deteriorates rather than wears out.
- McCall: operation, revision, transition.
- Waterfall: sequential, documents frozen. Spiral: four quadrants, risk-driven, Boehm 1986, meta-model.
- Prototype: throwaway; incremental: core first; evolutionary: versions guided by feedback.

⭐ Important questions

1. Define software engineering. Explain the characteristics of software and the software crisis.
2. Compare software engineering with conventional engineering. Explain software quality attributes.
3. Explain the waterfall model with a diagram, advantages and disadvantages.
4. Explain the spiral model. Why is it called a meta-model?
5. Explain the prototyping, evolutionary and iterative enhancement models and compare all SDLC models.

NEXT ENGINEER
sheerat

UNIT 2 Software Requirement Specifications and Quality Assurance

✍️ 2-mark definitions

- **Requirement:** a condition or capability the system must have to meet a user's need or a contract.
- **Functional requirement:** what the system must do (for example "the system shall generate a monthly salary slip").
- **Non-functional requirement:** how well the system must do it: performance, security, reliability, usability, portability (for example "the page shall load within 2 seconds").
- **Requirement engineering:** the process of finding, analysing, documenting, validating and managing requirements.
- **Elicitation:** gathering requirements from stakeholders through interviews, questionnaires, brainstorming, facilitated application specification technique (FAST), quality function deployment (QFD) and use cases.
- **Feasibility study:** an analysis of whether the project is worth doing: technical, economic (cost-benefit), operational, schedule and legal feasibility.
- **SRS (Software Requirements Specification):** the document that fully describes what the software will do and how it is expected to perform; it is the basis of the contract between customer and developer.
- **Data Flow Diagram (DFD):** a graphical model showing how data flows through processes, data stores and external entities.
- **Data dictionary:** a repository describing every data item in the DFDs (name, aliases, type, range, where used).
- **Decision table:** a table that lists conditions and actions and shows which actions to take for each combination of conditions.
- **Verification:** "Are we building the product right?" Checking that each phase's output meets its specification (reviews, inspections).
- **Validation:** "Are we building the right product?" Checking that the final software meets the user's needs (testing).
- **SQA (Software Quality Assurance):** planned and systematic activities to make sure the process and product meet quality standards.

💡 Core concepts

Requirement engineering process

1. **Feasibility study** → feasibility report.
2. **Elicitation:** interviews, questionnaires, observation, brainstorming, FAST, QFD (normal, expected and exciting requirements), use cases.
3. **Analysis:** remove conflicts, ambiguity and incompleteness; model with DFD, ER diagram, decision tables, use case diagrams; prioritise.

4. **Documentation:** write the SRS.
5. **Review (validation):** check the SRS for correctness, completeness, consistency and testability through reviews, prototypes and test-case generation.
6. **Management:** track and control changes to requirements over the project (traceability matrix, change requests).

Data Flow Diagrams

- Symbols (Yourdon/DeMarco): circle or bubble = process; arrow = data flow; open rectangle or two parallel lines = data store; rectangle = external entity (source or sink).
- **Levels:** Level 0 (context diagram) shows the whole system as one process with its external entities. Level 1 breaks it into main processes. Level 2 breaks each further.
- **Rules:** every process has at least one input and one output; data cannot flow directly between two stores or two external entities without a process; balancing: inputs and outputs of a parent process must match its child diagram.
- **Example (library system, Level 0):** Member → (book request, return) → [Library System] → (issue slip, fine notice) → Member; Librarian → (new book details) → [Library System] → (reports) → Librarian.
- Level 1 processes: 1.0 Register member, 2.0 Issue book, 3.0 Return book and calculate fine, 4.0 Generate reports; data stores: Members, Books, Transactions.

ER diagram (information modelling): entities (rectangles), attributes (ovals; key underlined), relationships (diamonds) with cardinality (1:1, 1:N, M:N). Example: Member (MemberID, Name) —borrows (M:N, IssueDate)— Book (ISBN, Title).

Decision table example (discount policy)

	Rule 1	Rule 2	Rule 3	Rule 4
C1: Order above ₹5,000?	Y	Y	N	N
C2: Regular customer?	Y	N	Y	N
A1: 15% discount	X			
A2: 10% discount		X	X	
A3: No discount				X

With n conditions there are 2^n rules (here $2^2 = 4$).

SRS: characteristics of a good SRS (IEEE 830) Correct, unambiguous, complete, consistent, ranked for importance and stability, verifiable, modifiable, traceable. It should state what the system does, not how.

IEEE 830 SRS outline

1. Introduction: purpose, scope, definitions, references, overview.
2. Overall description: product perspective, product functions, user characteristics, constraints, assumptions and dependencies.
3. Specific requirements: external interfaces (user, hardware, software, communication), functional requirements, performance requirements, design constraints, software system attributes (reliability, availability, security, maintainability, portability), other requirements.
4. Appendices and index.

SQA: verification and validation

- Verification activities: reviews, walkthroughs, inspections of documents and code (static).
- Validation activities: testing the executable software (dynamic): unit, integration, system, acceptance testing.
- **V-model:** each development phase on the left (requirements, system design, architecture, module design) is paired with a testing phase on the right (acceptance, system, integration, unit testing). Test planning starts early.

SQA plan (IEEE 730): purpose, reference documents, management (organization and responsibilities), documentation, standards and metrics, reviews and audits, testing, problem reporting and corrective action, tools, code control, media control, supplier control, records, training and risk management.

Quality frameworks

- **ISO 9000:** a family of international standards for quality management systems. ISO 9001 applies to organizations that design, develop, produce and service products (including software); ISO 9000-3 gave guidelines for applying ISO 9001 to software. It requires a documented quality system, management responsibility, document control, design control, inspection and testing, corrective action, internal audits and training. Certification is by an external registrar.
- **SEI-CMM (Capability Maturity Model, Software Engineering Institute, Carnegie Mellon):** five maturity levels of an organization's software process:

Level	Name	Key process areas / description
1	Initial	Ad hoc, chaotic; success depends on individual heroes
2	Repeatable	Basic project management: requirements management, project planning, project tracking, subcontract management, SQA, configuration management
3	Defined	Organization-wide standard process: process definition and focus, training, integrated software management, peer reviews, inter-group coordination
4	Managed	Quantitative process management and software quality management with metrics
5	Optimizing	Continuous process improvement: defect prevention, technology change management, process change management

- **ISO 9000 vs SEI-CMM:** ISO 9000 is a general standard for any industry with pass/fail certification; CMM is specific to software and gives five levels for gradual improvement. CMM was later replaced by CMMI (integrated, covering development, services and acquisition).

🧮 Solved examples

Example 1 (functional vs non-functional). Online exam system:

- Functional: register student; display questions; auto-submit when time ends; compute the score.
- Non-functional: support 5,000 users at once; 99.9% availability during exams; passwords stored hashed; works on mobile browsers.

Example 2 (decision table for leave approval). Conditions: C1 leave ≤ 3 days, C2 leave balance available. Actions: approve by manager (Y, Y), refer to HR (N, Y), reject (Y, N) and (N, N). Four rules from 2^2 combinations.

Key points to remember

- Requirement engineering: feasibility, elicitation, analysis, documentation, review, management.
- DFD symbols; Level 0 = context diagram; balancing rule.
- Good SRS: correct, unambiguous, complete, consistent, ranked, verifiable, modifiable, traceable.
- Verification = right product build (static); validation = right product (dynamic).
- CMM levels: Initial, Repeatable, Defined, Managed, Optimizing.

⭐ Important questions

1. Explain the requirement engineering process with each step.
2. What is a feasibility study? Explain its types.
3. Draw Level 0 and Level 1 DFDs for a library / hospital / ATM system. Explain DFD rules and the data dictionary.
4. What is an SRS? Explain the characteristics of a good SRS and the IEEE 830 format.
5. Differentiate verification and validation. Explain SQA and the SQA plan.
6. Explain the SEI-CMM levels. Compare ISO 9000 and SEI-CMM.

NEXT ENGINEER _sheerat_

UNIT 3 Software Design and Metrics

✍️ 2-mark definitions

- **Software design:** the process of converting the SRS into a blueprint of the system: its architecture, modules, interfaces and data.
- **Abstraction:** focusing on the essential features while hiding details (procedural and data abstraction).
- **Modularity:** dividing software into separately named, addressable modules that are integrated to satisfy requirements.
- **Information hiding:** each module hides its internal details (data structures, algorithms) from others and exposes only an interface.
- **Architectural design:** the high-level structure of the system: its major components and how they interact (styles: layered, client-server, pipe-and-filter, repository, MVC, microservices).
- **Coupling:** the degree of interdependence between modules. Good design has **low coupling**.
- **Cohesion:** the degree to which the elements inside a module belong together. Good design has **high cohesion**.
- **Structure chart:** a tree diagram showing the modules of a system, their hierarchy, and the data and control passed between them.
- **Pseudo code:** a structured, English-like description of an algorithm without programming-language syntax.
- **Software metric:** a quantitative measure of a property of software or its process.
- **Function point (FP):** a measure of software size based on the functionality delivered to the user, independent of the language.
- **Cyclomatic complexity:** the number of linearly independent paths through a program, computed from its control flow graph.

💡 Core concepts

Design concepts: abstraction, architecture, patterns, separation of concerns, modularity, information hiding, functional independence (coupling and cohesion), refinement (top-down elaboration), refactoring, design classes.

Types of coupling (worst to best)

Coupling	Meaning
Content (worst)	One module directly changes or jumps into the internal code or data of another
Common	Modules share global data
External	Modules share an externally imposed format, device or protocol
Control	One module passes a control flag that decides the logic of another
Stamp	A whole data structure is passed though only part is used

Coupling	Meaning
Data (best)	Only the needed elementary data items are passed as parameters

Types of cohesion (worst to best)

Cohesion	Meaning
Coincidental (worst)	Unrelated tasks grouped by chance
Logical	Tasks of a similar kind (for example all input routines), selected by a flag
Temporal	Tasks done at the same time (for example initialization)
Procedural	Tasks done in a fixed order of steps
Communicational	Tasks that operate on the same data
Sequential	Output of one task is the input of the next
Functional (best)	All elements contribute to a single, well-defined function

Function-oriented vs object-oriented design

Function-oriented	Object-oriented
System seen as a set of functions transforming data (DFD, structure chart; SA/SD)	System seen as interacting objects with state and behaviour (UML class, sequence diagrams)
Data is often global and shared	Data is encapsulated inside objects
Top-down decomposition of functions	Identify classes, relationships, inheritance
Suits computation-heavy systems	Suits complex systems; better reuse and maintenance

Top-down vs bottom-up design: top-down starts with the whole system and refines it into smaller modules (good when the problem is well understood; needs stubs to test). Bottom-up starts with basic, reusable modules and combines them (good for reuse; needs drivers to test). Most real projects combine both (sandwich approach).

Structure chart symbols: rectangle = module; arrow between modules = call; small arrow with empty circle = data couple; small arrow with filled circle = control flag; diamond = conditional call; curved arrow = loop (repeated calls). Transform analysis turns a DFD into a structure chart by finding input (afferent), central transform and output (efferent) parts.

Size metrics: LOC

- Lines of code (excluding comments and blank lines), measured in KLOC. Used for productivity (LOC per person-month) and quality (defects per KLOC).
- Drawbacks: depends on language and programmer style; cannot be known until coding is done; penalizes well-designed short programs.

Halstead's software science

- n1 = number of distinct operators; n2 = number of distinct operands; N1 = total occurrences of operators; N2 = total occurrences of operands.
- Vocabulary n = n1 + n2; program length N = N1 + N2; estimated length $\hat{N}$ = n1 log2 n1 + n2 log2 n2.

- Volume V = N log2 n (bits); difficulty D = (n1/2) × (N2/n2); effort E = D × V; time T = E/18 seconds (Stroud number 18); program level L = 1/D.

Function point analysis (Albrecht)

Component	Simple	Average	Complex
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

- UFP = Σ (count × weight).
- 14 general system characteristics (data communication, distributed processing, performance, heavily used configuration, transaction rate, online data entry, end-user efficiency, online update, complex processing, reusability, installation ease, operational ease, multiple sites, facilitate change), each rated 0 to 5. Total degree of influence ΣFi lies between 0 and 70.
- CAF (VAF) = 0.65 + 0.01 × ΣFi, ranging from 0.65 to 1.35.
- **FP = UFP × CAF.**

Cyclomatic complexity (McCabe)

- Draw the control flow graph (CFG): nodes are statements or blocks; edges are transfers of control.
- V(G) = E − N + 2 (for one connected component; E − N + 2P in general) = number of predicate (decision) nodes + 1 = number of regions of the planar graph (including the outer region).
- V(G) gives the number of independent paths, which is the minimum number of test cases for basis path testing. A value above 10 suggests the module should be simplified.

🧮 Solved examples

Example 1 (function points). A system has 24 EI, 16 EO, 22 EQ, 4 ILF and 2 EIF, all of average complexity, and ΣFi = 42.

- UFP = 24×4 + 16×5 + 22×4 + 4×10 + 2×7 = 96 + 80 + 88 + 40 + 14 = **318**.
- CAF = 0.65 + 0.01 × 42 = **1.07**.
- FP = 318 × 1.07 = **340.26**.

Example 2 (Halstead). A program has n1 = 10, n2 = 8, N1 = 40, N2 = 30.

- n = 18; N = 70; $\hat{N}$ = 10 log2 10 + 8 log2 8 = 33.22 + 24 = **57.22**.
- V = 70 × log2 18 = 70 × 4.17 = **291.9 bits**.
- D = (10/2) × (30/8) = **18.75**; E = 18.75 × 291.9 ≈ **5,473**; T = 5473/18 ≈ **304 seconds** (about 5 minutes).

Example 3 (cyclomatic complexity).

NEXT ENGINEER _sheerat_

```
int classify(int a, int b) {
    int r = 0;          /* node 1 (with the if test) */
    if (a > b)
        r = 1;          /* node 2 */
    else
        r = 2;          /* node 3 */
    while (a > 0)       /* node 4 */
        a--;            /* node 5 */
    return r;           /* node 6 */
}
```

- Edges: 1→2, 1→3, 2→4, 3→4, 4→5, 5→4, 4→6, so E = 7 and N = 6.
- V(G) = 7 − 6 + 2 = **3**. Check: 2 decisions (if, while) + 1 = 3; regions = 3.
- Independent paths: 1-2-4-6; 1-3-4-6; 1-2-4-5-4-6. So at least **3 test cases**, for example (a=0, b=−1), (a=0, b=1), (a=1, b=0).

Example 4 (coupling and cohesion). A module `processOrder(order, flag)` that prints an invoice if flag = 1 and emails it if flag = 2 shows **control coupling** and **logical cohesion**. Better: two separate modules `printInvoice(order)` and `emailInvoice(order)`, each with data coupling and functional cohesion.

Formulas to remember

- Halstead: n = n1 + n2; N = N1 + N2; $\hat{N}$ = n1 log2 n1 + n2 log2 n2; V = N log2 n; D = (n1/2)(N2/n2); E = D × V; T = E/18.
- FP = UFP × (0.65 + 0.01 ΣFi).
- V(G) = E − N + 2P = P(decisions) + 1 = regions.

⭐ Important questions

1. Explain the basic concepts of software design: abstraction, modularity, information hiding, refinement.
2. Define coupling and cohesion. Explain their types and why low coupling and high cohesion are desirable.
3. Compare function-oriented and object-oriented design; top-down and bottom-up design. Explain structure charts.
4. Explain Halstead's software science measures with a numerical.
5. Explain function point analysis and calculate FP for given data.
6. Define cyclomatic complexity. Draw the CFG of a given program and find V(G) by three methods.

UNIT 4 Software Testing

✍️ 2-mark definitions

- **Testing:** executing a program with the intent of finding errors (Myers). A successful test is one that finds an undiscovered error.
- **Error, fault and failure:** an error is a human mistake; it causes a fault (defect, bug) in the code; a fault, when executed, may cause a failure (the system does not do what it should).
- **Test case:** a set of inputs, execution conditions and expected outputs, designed to test a particular objective.
- **Test suite:** a collection of test cases. **Test data:** the input values used.
- **Unit testing:** testing individual modules in isolation.
- **Integration testing:** testing the interfaces between modules as they are combined.
- **System testing:** testing the complete, integrated system against the SRS (functional and non-functional).
- **Acceptance testing:** testing by the customer to decide whether to accept the system.
- **Regression testing:** re-running earlier tests after a change to make sure nothing that worked before is broken.
- **Stub:** a dummy called module that replaces a lower-level module not yet developed (used in top-down integration).
- **Driver:** a dummy calling module that calls the module under test and passes test data (used in bottom-up integration).
- **Alpha testing:** acceptance testing done by users at the developer's site in a controlled environment.
- **Beta testing:** testing by real users at their own sites, in a live environment, before release.
- **Static testing:** examining code or documents without executing them (reviews, walkthroughs, inspections).

💡 Core concepts

Objectives of testing: find defects; check that requirements are met; build confidence in quality; prevent defects through early reviews; give information for decisions. Testing can show the presence of bugs, never their absence (Dijkstra).

Levels of testing: unit → integration → system → acceptance. Regression testing is done at every level after changes.

Integration strategies

Strategy	How it works	Needs	Advantage	Disadvantage
Big bang	All modules combined at once	Nothing extra	Simple for small systems	Hard to locate faults

Strategy	How it works	Needs	Advantage	Disadvantage
Top-down	Start from the main module, add lower ones (depth-first or breadth-first)	Stubs	Early working skeleton; major control flaws found early	Many stubs; lower-level details tested late
Bottom-up	Start from the lowest modules, combine into clusters, move up	Drivers	Easy to create test conditions; no stubs	No working program until the end
Sandwich (hybrid)	Top-down for upper layers, bottom-up for lower layers	Stubs and drivers	Combines both advantages	More complex to manage

Functional and performance testing (system testing)

- Functional testing checks every function against the SRS.
- Performance testing types: load testing (expected load), stress testing (beyond limits until failure), volume testing (large data), security testing, recovery testing, configuration and compatibility testing, usability testing.

White-box vs black-box testing

White-box (structural, glass-box)	Black-box (functional, behavioural)
Tester knows the internal code	Tester knows only the specification
Based on code structure: statement, branch, condition, path coverage; basis path testing; loop testing; data flow testing	Based on inputs and outputs: equivalence partitioning, boundary value analysis, cause-effect graphing, decision table testing
Done mainly by developers at unit level	Done at all levels, often by independent testers
Finds logic and path errors; cannot find missing features	Finds missing or wrong functions and interface errors; cannot test all paths

Coverage criteria (white-box): statement coverage (every statement executed at least once) < branch/decision coverage (every branch true and false) < condition coverage < path coverage (every independent path).

Black-box techniques

- **Equivalence class partitioning:** divide the input domain into classes whose members are expected to behave the same; pick one test from each valid and invalid class.
- **Boundary value analysis (BVA):** errors cluster at boundaries; test at min, min+1, nominal, max−1, max (and min−1, max+1 for robustness). For n variables, basic BVA gives 4n + 1 test cases; robust BVA gives 6n + 1.
- **Cause-effect graphing:** link input conditions (causes) to outputs (effects) with Boolean logic, then convert into a decision table.

Alpha vs beta testing

Alpha	Beta
At the developer's site	At the user's site
Controlled environment, developer present	Real environment, developer absent

Alpha	Beta
Done before beta	Done after alpha, just before release
Finds bugs early with quick fixes	Gets feedback from many real users

Static testing techniques

- **Formal Technical Review (FTR):** a structured meeting (3 to 5 people, about 2 hours, prepared in advance) to find errors in a work product. Roles: producer, review leader, reviewers, recorder. Outcome: accept, reject, or accept with minor changes; a review summary report and issues list.
- **Walkthrough:** the author leads the team through the document or code, explaining it step by step; informal; aims at understanding and finding errors.
- **Code inspection (Fagan):** the most formal review; trained moderator; checklist of common errors (uninitialized variables, off-by-one loops, mismatched parameters, array bounds, incorrect logic); phases: planning, overview, preparation, inspection meeting, rework, follow-up. Metrics are recorded.
- **Compliance with design and coding standards:** checking naming conventions, comments, indentation, limits on module length and nesting, error handling, and banned constructs; often automated with static analysis tools (linters).

🧮 Solved examples

Example 1 (equivalence classes and BVA). A field accepts an age from 18 to 60.

- Equivalence classes: invalid < 18 (test 10), valid 18 to 60 (test 35), invalid > 60 (test 70).
- BVA (single variable, 4n + 1 = 5): 18, 19, 39, 59, 60. Robust BVA adds 17 and 61 (6n + 1 = 7).

Example 2 (BVA with two variables). x in [1, 100], y in [1, 50]. Basic BVA gives 4 × 2 + 1 = **9** test cases: hold one variable at its nominal value and take the five boundary values of the other, sharing the case (nominal, nominal). Example: (50, 1), (50, 2), (50, 49), (50, 50), (1, 25), (2, 25), (99, 25), (100, 25), (50, 25).

Example 3 (stubs and drivers). Module A calls B and C. Testing A first (top-down) needs stubs for B and C that return fixed values. Testing B first (bottom-up) needs a driver that plays the role of A, calls B with test data and prints results.

Key points to remember

- Error → fault → failure. Testing shows presence, not absence, of bugs.
- Top-down uses stubs; bottom-up uses drivers.
- White-box: coverage, basis path (V(G) tests). Black-box: equivalence classes, BVA (4n + 1, robust 6n + 1), cause-effect graph.
- Alpha at developer's site; beta at user's site.
- Static: FTR, walkthrough, inspection (most formal), standards compliance.

⭐ Important questions

1. Define testing and its objectives. Differentiate error, fault and failure.

2. Explain unit, integration, system, acceptance and regression testing.
3. Explain top-down and bottom-up integration testing with stubs and drivers.
4. Compare white-box and black-box testing. Explain equivalence partitioning and boundary value analysis with examples.
5. Differentiate alpha and beta testing. Explain functional and performance testing.
6. Explain static testing: formal technical review, walkthrough and code inspection.

NEXT ENGINEER
sheerat

UNIT 5 Software Maintenance and Project Management

✍️ 2-mark definitions

- **Software maintenance:** modifying software after delivery to correct faults, improve performance or other attributes, or adapt it to a changed environment (IEEE).
- **Software as an evolutionary entity:** useful software must keep changing or it becomes less useful (Lehman's laws of software evolution: continuing change and increasing complexity).
- **Corrective maintenance:** fixing bugs found after delivery.
- **Adaptive maintenance:** changing software to work in a new environment (new OS, hardware, law, tax rules).
- **Perfective maintenance:** adding features or improving performance and maintainability as users request.
- **Preventive maintenance:** changes that prevent future problems, such as restructuring code and updating documentation.
- **Reverse engineering:** analysing existing software to recover its design and specification (going from code to higher-level abstractions).
- **Re-engineering:** reverse engineering followed by forward engineering: restructuring or rewriting an old system to improve it without changing its functions.
- **Software Configuration Management (SCM):** the discipline of identifying, controlling, recording and auditing changes to software work products (configuration items) during the life cycle.
- **Baseline:** a reviewed and approved version of a work product that can be changed only through formal change control.
- **CASE tools:** Computer-Aided Software Engineering tools that automate SDLC activities: upper CASE (analysis and design, for example UML modelling tools), lower CASE (coding, testing, debugging) and integrated CASE.
- **Risk management:** identifying, analysing, planning for and monitoring risks to reduce their effect on the project.

💡 Core concepts

Need for maintenance: fixing errors, changing user requirements, new hardware and OS, new laws and policies, performance improvement, migration of legacy software, retirement of old technology. Maintenance typically consumes 60% or more of total life-cycle cost.

Distribution of maintenance effort (commonly cited figures): perfective about 50%, adaptive about 25%, corrective about 21%, preventive about 4%.

Cost of maintenance

- Factors: age and structure of the program, documentation quality, team stability, programming language, configuration management, domain complexity.
- **Belady and Lehman model:** M = P + K·e^(c – d), where M is total maintenance effort, P is productive effort, K is an empirical constant, c is complexity due to lack of structured design and documentation,

and d is the degree of familiarity with the software. Effort grows exponentially when complexity is high and familiarity is low.

- **Boehm's model:** Annual Change Traffic (ACT) = (KLOC added + KLOC deleted)/total KLOC. Annual maintenance effort = ACT × development effort (in person-months).

Re-engineering process model (Pressman): inventory analysis → document restructuring → reverse engineering → code restructuring → data restructuring → forward engineering. Reverse engineering levels: processing (code), data (data structures, database), user interface.

Software Configuration Management

- **Configuration items:** SRS, design documents, source code, test cases, user manuals, tools.
- **SCM tasks:** identification of items, version control, change control, configuration auditing, status reporting.
- **Change control process:** change request submitted → evaluated (technical merit, side effects, cost) → change control authority (CCA or board) approves or rejects → engineering change order → item checked out, changed, reviewed → checked in → new baseline; record in the change log.
- **Version control:** manages multiple versions of files; check-in/check-out, branching and merging, history and rollback. Tools: Git, Subversion (SVN), CVS, Mercurial. A version is a new instance with changes; a variant is an alternative version for a different platform.

Software project management: estimation

- Steps: estimate size (LOC or FP) → effort (person-months) → schedule (months) → cost.
- **COCOMO (Constructive Cost Model, Boehm, 1981):** three project types:
- Organic: small team, familiar problem, flexible requirements (for example payroll).
- Semi-detached: medium size, mixed experience (for example compilers, database systems).
- Embedded: tight hardware, software and operational constraints (for example air traffic control, avionics).

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

- **Basic COCOMO:** Effort E = a (KLOC)^b person-months; development time D = c (E)^d months; average staff = E/D; productivity = KLOC/E.
- **Intermediate COCOMO:** E = a (KLOC)^b × EAF, with a = 3.2, 3.0, 2.8 for the three modes and EAF the product of 15 cost drivers (product attributes such as reliability and complexity; hardware attributes such as time and memory constraints; personnel attributes such as analyst and programmer capability; project attributes such as use of tools and schedule).
- **Detailed COCOMO:** applies the cost drivers phase by phase and module by module.
- **COCOMO II:** for modern development: application composition, early design and post-architecture models.

Resource allocation models

- **Putnam (SLIM) / Rayleigh-Norden curve:** staffing on a project follows a Rayleigh curve: it rises, peaks around the time of design and coding, and falls. Putnam's software equation: L = Ck K^(1/3)

td^(4/3), where L is size, K total effort, td development time and Ck a technology constant. Shortening the schedule increases effort sharply (effort $\propto 1/td^4$).

- **Brooks' law:** adding people to a late project makes it later (because of training and communication overhead; with n people, n(n − 1)/2 communication paths).
- **Scheduling tools:** work breakdown structure (WBS), Gantt chart, PERT and CPM (critical path), milestones.

Risk analysis and management

- **Types of risk:** project risks (schedule, budget, staff, resources), technical risks (design, implementation, new technology), business risks (market, strategy, management support). Also known, predictable and unpredictable risks.
- **Process:** (1) risk identification (checklists, brainstorming); (2) risk analysis (probability and impact; risk exposure RE = P × C, where C is the cost if the risk occurs); (3) risk planning: avoidance, transfer (insurance, outsourcing), mitigation (reduce probability or impact), acceptance with contingency; (4) risk monitoring (RMMM plan: Risk Mitigation, Monitoring and Management).
- Boehm's top risks: personnel shortfalls, unrealistic schedules and budgets, developing the wrong functions, wrong user interface, gold-plating, continuing requirement changes, shortfalls in externally supplied components, real-time performance shortfalls.

🧮 Solved examples

Example 1 (Basic COCOMO for a 32 KLOC project).

- Organic: E = 2.4 × 32^1.05 = **91.3 PM**; D = 2.5 × 91.3^0.38 = **13.9 months**; staff = 91.3/13.9 ≈ **6.6 people**.
- Semi-detached: E = 3.0 × 32^1.12 = **145.5 PM**; D = 2.5 × 145.5^0.35 = **14.3 months**; staff ≈ **10.2**.
- Embedded: E = 3.6 × 32^1.20 = **230.4 PM**; D = 2.5 × 230.4^0.32 = **14.3 months**; staff ≈ **16.2**.
- Note: the same size needs much more effort as the project becomes more constrained, but the schedule hardly changes.

Example 2 (Intermediate COCOMO). Organic project of 32 KLOC with high reliability (1.15) and high analyst capability (0.91), all other drivers nominal (1.00).

- EAF = 1.15 × 0.91 = **1.0465**.
- E = 3.2 × 32^1.05 × 1.0465 ≈ **127.4 PM**.

Example 3 (risk exposure). A key developer may leave with probability 0.3; replacing them would cost 4 person-months at ₹1.5 lakh each = ₹6 lakh. RE = 0.3 × 6 = **₹1.8 lakh**. Mitigation: pair programming and documentation so knowledge is shared.

Example 4 (Boehm maintenance). Development effort 600 PM; in a year 3 KLOC added and 2 KLOC deleted from a 50 KLOC system. ACT = 5/50 = 0.1. Annual maintenance effort = 0.1 × 600 = **60 PM**.

Formulas to remember

- Basic COCOMO: E = a(KLOC)^b; D = c(E)^d; staff = E/D.
- Intermediate: E = a(KLOC)^b × EAF.
- Risk exposure RE = P × C.

- ACT = (added + deleted)/total; maintenance effort = ACT × development effort.
- Brooks: communication paths = n(n − 1)/2.

⭐ Important questions

1. What is software maintenance? Explain its types and the need for maintenance. Discuss the cost of maintenance.
2. Differentiate reverse engineering and re-engineering. Explain the re-engineering process.
3. What is software configuration management? Explain change control and version control.
4. Explain CASE tools and their types.
5. Explain the COCOMO model (basic and intermediate) and solve a numerical.
6. Explain the Putnam resource allocation model.
7. What is risk? Explain risk identification, analysis, planning and monitoring (RMMM).

Last-hour checklist

☐ Software characteristics; software crisis; SE vs conventional engineering; McCall's quality factors.
☐ Waterfall, prototype, spiral (four quadrants), evolutionary, incremental: diagram, pros, cons, comparison table.
☐ Requirement engineering steps; feasibility types; functional vs non-functional.
☐ DFD symbols, levels and rules; ER diagram; decision table (2^n rules); data dictionary.
☐ Good SRS characteristics; IEEE 830 outline.
☐ Verification vs validation; V-model; SQA plan; ISO 9000; five CMM levels.
☐ Coupling (content to data) and cohesion (coincidental to functional) order.
☐ Function-oriented vs OO; top-down vs bottom-up; structure chart symbols.
☐ Halstead (V = 291.9 example), FP (340.26 example), V(G) = 3 example.
☐ Testing levels; stubs vs drivers; integration strategies table.
☐ White-box vs black-box; equivalence classes; BVA 4n + 1; alpha vs beta.
☐ FTR, walkthrough, inspection.
☐ Maintenance types and effort split; Belady-Lehman; re-engineering vs reverse engineering.
☐ SCM, baseline, change control steps, version control.
☐ COCOMO table (32 KLOC: 91.3, 145.5, 230.4 PM); Putnam; risk exposure.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Software Engineering: A Practitioner's Approach	Roger S. Pressman	McGraw Hill	WorldCat
2	Software Engineering	Pankaj Jalote	Wiley	WorldCat
3	Fundamentals of Software Engineering	Rajib Mall	PHI Learning	WorldCat
4	Software Engineering	K. K. Aggarwal, Yogesh Singh	New Age International	WorldCat
5	Fundamentals of Software Engineering	Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli	PHI Learning	WorldCat
6	Software Engineering	Ian Sommerville	Addison Wesley / Pearson	WorldCat
7	Software Engineering	Kassem Saleh	Cengage Learning	WorldCat
8	Software Engineering	Shari Lawrence Pfleeger	Macmillan / Pearson	WorldCat

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER _sheerat_